

AeroHaptix re-run: MultiCBF haptic rendering for UAV teleoperation, re-implemented, verified and stress-tested in simulation

Jerry (Qilong) Cheng

September 2026

Abstract

AeroHaptix [1] is a wearable vibrotactile system that renders obstacle directions on the upper body during UAV teleoperation: 32 voice-coil units whose positions came from a perceptual study, and MultiCBF, a rendering rule that assigns one exponential control barrier function (ECBF) to every obstacle, computes a local safe input per violated barrier and drives the actuator facing the obstacle with an intensity proportional to the size of the correction. This document records a re-implementation of the released algorithm (super-ellipsoid, pillar and half-space barriers, the ECBF condition, an exact solver for the global safe input in $\mathbb{R}^3$, the cue-rendering rule and the released 32-direction table), its verification, and a simulated re-run of the paper’s tunnel study with a modelled operator in place of a human. With 24 tunnels per flying direction, no feedback gives 2.92, 5.71 and 5.92 collisions per flight in the forward, right and upward tunnels; the cues reduce these to 0.54, 0.67 and 0.83 and the input disagreement from 6.73 to 3.54 m/s². Sweeps over the actuator count, the operator’s reaction delay and gain, the barrier gains and padding and the camera field of view show that the benefit depends far more on the operator’s reaction than on the algorithm’s constants, that 32 directions capture most of what an exact direction gives, and that the barrier gain of the paper sits at the knee of the collision curve. All figures are regenerated by `experiments/make_figures.py` from `out/study.json` (1332 flights).

Keywords: haptic teleoperation, vibrotactile feedback, control barrier functions, UAV, shared control, simulation study.

1 Overview

The paper’s contribution has three parts: custom chained hardware with low-latency control of many actuators; an actuator layout learned from a perceptual study so that obstacle directions are covered uniformly on the body; and MultiCBF, which extends the single-CBF haptic teleoperation of Zhang et al. [3] to several simultaneous obstacle cues. Its user study (12 participants, three feedback conditions, three flying directions in a simulated tunnel) found fewer collisions and less input disagreement with the suit than without feedback, higher haptic usefulness than a force-feedback joystick, and no added workload.

This report does not repeat the human study. It re-implements the algorithm from the released code [2] and runs it, in a lightweight simulator of the same tunnel recipe, with a modelled operator whose perception and reactions are explicit parameters. The purpose is (i) to have a verified, reusable implementation, (ii) to show the mechanism quantitatively (which actuators fire, how often, how early), and (iii) to learn which knobs the benefit is sensitive to. The same implementation, ported to JavaScript, drives the interactive suit, the barrier playground and the 3-D flight replays

of the web page aerohaptix.html. Section 2 summarises the system as published; the numbers there are the paper’s, not ours.

2 The system as published

Hardware. Each vibration unit is a PUI Audio HD-VA3222 voice-coil actuator (32 mm × 22 mm, 80 Hz to 500 Hz, 2.52 g peak-to-peak at 133 Hz and 1.5 V_{rms}) on a custom board with a PIC16F18313 microcontroller and a DRV8837 H-bridge, giving 16 intensity levels, 4 frequencies and 2 waveforms. Units form chains behind a central unit and a power converter; a two-byte UART message at 115.2 kbaud carries a target address that every unit decrements before forwarding, 125 µs per hop, 2.5 ms across 20 units. The released direction table lists 32 motor ids on four buses (addresses 0, 30, 120, 150) holding 5, 7, 8 and 12 units.

Layout. Ten participants wore a 46-unit grid (18 front, 18 back, 3 per side of the waist, 2 per shoulder, 8 cm apart) in VR and pointed at the direction they felt each unit from, 230 trials each. An MLP (five hidden layers of 64, ReLU, total-variation regularisation) learned the inverse map from a direction $(\vartheta, \varphi) \in S^2$ to a body position. Thirty-two directions, $\vartheta \in \{\pi/6, \pi/3, \pi/2, 2\pi/3\}$ and eight azimuths, were pushed through it; $\vartheta = 5\pi/6$ was dropped because nobody could feel “below” on the upper body. The released table confirms exactly this grid (Section 3.8).

Study. Twelve participants (11 male, mean age 24, seven with UAV experience) flew an AirSim quadrotor [7] through $5 \times 5 \times 50$ m tunnels with four bounding planes and fifteen random cubes, spheres and cylinders, in three conditions (NA: Xbox controller, no feedback; FSC: Novint Falcon force joystick with single-CBF force; VSC: Xbox controller plus AeroHaptix with MultiCBF) and three flying directions (forward, right, upward; the camera always faces $+x$). Reported per flight (values read from the paper’s Fig. 7): collisions NA 7.5, FSC 4.2, VSC 5.2; by direction FWD 2.0, R 7.9, UP 7.0; input disagreement NA 3.55, FSC 2.8, VSC 2.0 m/s²; distance 57 m to 75 m. Feedback affected collisions ($F(2, 22) = 8.10$, $p < 0.01$) and disagreement ($F(2, 22) = 4.80$, $p < 0.05$); direction affected collisions ($F(2, 22) = 15.65$, $p < 0.001$). VSC scored 82/100 on haptic usefulness against 57 for FSC ($\chi^2(2) = 21.4$, $p < 0.001$) and let participants report more felt-only obstacles ($F(1, 11) = 7.44$, $p < 0.05$). Figure 1 redraws these values.

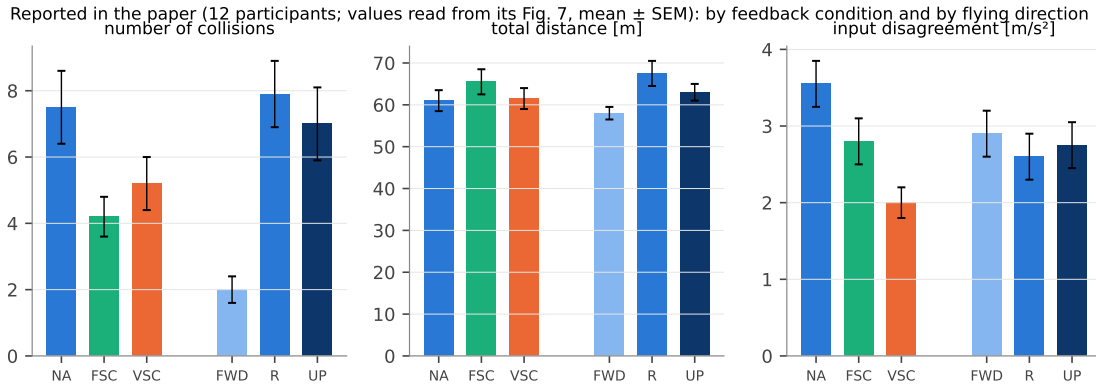


Figure 1: The paper’s objective results (mean $\pm$ SEM over 12 participants), read from its Fig. 7 to about ± 0.2 : by feedback condition (NA, FSC, VSC) and by flying direction (FWD, R, UP).

3 Method

3.1 Barriers and the exponential CBF condition

The UAV is a double integrator with state $\mathbf{x} = [\mathbf{q}^\top, \mathbf{v}^\top]^\top \in \mathbb{R}^6$ and input $\mathbf{u} \in \mathbb{R}^3$, $\dot{\mathbf{q}} = \mathbf{v}$, $\dot{\mathbf{v}} = \mathbf{u}$. Every obstacle b_i gets a barrier $h_i : \mathbb{R}^3 \rightarrow \mathbb{R}$ with $h_i(\mathbf{q}) \geq 0$ outside its safety boundary:

$$\text{plane: } h(\mathbf{q}) = \hat{\mathbf{n}} \cdot (\mathbf{q} - \mathbf{p}), \quad (1)$$

$$\text{cube } (n = 4), \text{ sphere } (n = 2): h(\mathbf{q}) = \sum_{k=1}^3 \left(\frac{q_k - c_k}{a} \right)^n - 1, \quad a = \text{size} + 0.717 \text{ m}, \quad (2)$$

$$\text{pillar along axis } \ell \text{ } (n = 2): h(\mathbf{q}) = \sum_{k \neq \ell} \left(\frac{q_k - c_k}{a} \right)^2 - 1. \quad (3)$$

The padding 0.717 m added to the obstacle's full size is the released code's `ScaleX + 0.717`; it covers the drone's own extent and, for cubes, the fact that a super-ellipsoid with $n = 4$ needs $a \geq 3^{1/4}s$ to contain a cube of half-side s . Writing $z_k = (q_k - c_k)/a$,

$$\nabla h = \frac{n}{a} z_k^{n-1}, \quad \dot{h} = \nabla h \cdot \mathbf{v}, \quad \ddot{h} = \underbrace{\sum_k \frac{n(n-1)}{a^2} z_k^{n-2} v_k^2}_{=: \kappa(\mathbf{q}, \mathbf{v})} + \nabla h \cdot \mathbf{u}. \quad (4)$$

Because $\mathbf{u}$ enters $\ddot{h}$ and not $\dot{h}$, the barrier has relative degree two and the exponential CBF condition of Nguyen and Sreenath [4] is used, as in the released code with $k_1 = k_2 = 6$:

$$\ddot{h} + k_2 \dot{h} + k_1 h \geq 0 \iff \underbrace{\nabla h(\mathbf{q}) \cdot \mathbf{u}}_{A_i} + \underbrace{\kappa(\mathbf{q}, \mathbf{v}) + k_2 \nabla h \cdot \mathbf{v} + k_1 h(\mathbf{q})}_{b_i} \geq 0. \quad (5)$$

For a plane $\kappa = 0$ and $A_i = \hat{\mathbf{n}}$. Each obstacle is considered only when $|\mathbf{q} - \mathbf{c}| < a + 5 \text{ m}$ (planes always). Validity: (5) renders the set $\{h \geq 0, \dot{h} + kh \geq 0\}$ forward invariant for the double integrator if the QP below is feasible; ∇h vanishes only at the obstacle centre, so the condition is well posed everywhere in the free space. Feasibility can fail inside a shell or between two shells that overlap; the solver then returns the least-violating candidate and flags it (Section 3.2). The gains are not tuned to the drone's true dynamics: the study applies $\mathbf{u}$ as a velocity increment (Section 3.4), so the class- $\mathcal{K}$ constants act on a mixed-unit quantity. We keep the convention.

3.2 The global safe input: an exact solver in $\mathbb{R}^3$

The global safe input is the projection of the operator's input on all in-range constraints,

$$\mathbf{u}_{\text{safe}} = \arg \min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{ref}}\|^2 \quad \text{s.t.} \quad A_i \mathbf{u} + b_i \geq 0 \quad \forall i \in \mathcal{I}(\mathbf{q}). \quad (6)$$

The released code solves (6) with SLSQP from a random start. With $\mathbf{u} \in \mathbb{R}^3$ the optimum is the least-norm correction onto the affine hull of its active set, which has at most three rows: $\mathbf{u}^* = \mathbf{u}_{\text{ref}} + A_{\mathcal{A}}^\top (A_{\mathcal{A}} A_{\mathcal{A}}^\top)^{-1} (-A_{\mathcal{A}} \mathbf{u}_{\text{ref}} - b_{\mathcal{A}})$. Our solver (`cbf.global.safe`) therefore enumerates candidates. Starting from the rows violated at $\mathbf{u}_{\text{ref}}$, it computes the corrections onto every single, pair and triple of the working set, adds any row a candidate violates, and repeats until a feasible candidate at distance R from $\mathbf{u}_{\text{ref}}$ is found. Then:

Lemma. If $\|\mathbf{u}^* - \mathbf{u}_{\text{ref}}\| \leq R$, every row j with $A_j \mathbf{u}_{\text{ref}} + b_j > \|A_j\| R$ is inactive at the optimum. *Proof.* $A_j \mathbf{u}^* + b_j \geq A_j \mathbf{u}_{\text{ref}} + b_j - \|A_j\| \|\mathbf{u}^* - \mathbf{u}_{\text{ref}}\| > 0$. $\square$

The remaining rows (typically two to four out of the nineteen constraints of a tunnel) are enumerated exhaustively and the closest feasible candidate is the exact optimum, because the optimum is one of the enumerated candidates and no feasible candidate can be closer. The cost is a handful of 3×3 solves per frame. `tests/test_cbf.py` checks 30 random feasible problems with eight rows against SLSQP at tolerance 10^{-12} : identical to 10^{-3} and never farther than SLSQP’s answer. If no feasible candidate exists after eight rounds (the drone is inside a shell), the candidate with the least squared violation is returned and the frame is flagged infeasible.

3.3 Local safe inputs and the cue-rendering rule (MultiCBF)

For every in-range obstacle whose constraint $\mathbf{u}_{\text{ref}}$ violates, $s_i = A_i \mathbf{u}_{\text{ref}} + b_i < 0$, the local safe input is the projection onto that single half-space, which is closed form,

$$\mathbf{u}_{\text{safe}}^{(i)} = \mathbf{u}_{\text{ref}} - \frac{s_i}{\|A_i\|^2} A_i, \quad \|\mathbf{u}_{\text{ref}} - \mathbf{u}_{\text{safe}}^{(i)}\| = \frac{|s_i|}{\|A_i\|}. \quad (7)$$

The released code obtains the same point with SLSQP. The actuator is the one whose direction $\hat{\mathbf{r}}_k$ (in the body frame; yaw is zero in the forward and right tunnels of the study) is most aligned with the direction to the obstacle’s centre (for a plane, $-\hat{\mathbf{n}}$), ties included; the duty is

$$I_i = \left\lfloor \min(15, K_v \|\mathbf{u}_{\text{ref}} - \mathbf{u}_{\text{safe}}^{(i)}\|) \right\rfloor, \quad K_v = 3 \text{ levels per m/s}^2, \quad (8)$$

and a cue is sent when $I_i \geq 1$. The flush keeps the two strongest actuators of the frame (`find_three_largest_indices` returns two), each at the strongest duty assigned to it. Frequency and waveform were fixed in the study (index 2 and 0).

3.4 The input convention of the released code

In `airsim_test.py` the operator’s stick gives a velocity command $\mathbf{v}_{\text{cmd}}$ (5 m/s at full deflection), the ECBF input is $\mathbf{u}_{\text{ref}} = \mathbf{v}_{\text{cmd}} - \mathbf{v}$, and the drone is commanded $\mathbf{v} + \mathbf{u}$ through AirSim’s velocity controller, $\mathbf{u} = \mathbf{u}_{\text{ref}}$ when assistance is off (the NA and VSC conditions) and $\mathbf{u} = \mathbf{u}_{\text{safe}}$ when it is on. Thus $\mathbf{u}$ is a velocity increment per 20 ms frame that the ECBF treats as an acceleration. Input disagreement, $\|\mathbf{u}_{\text{ref}} - \mathbf{u}_{\text{safe}}\|$, inherits these units; the paper labels it m/s^2 . We reproduce the convention exactly so that the numbers are comparable, and model the drone as a first-order velocity tracker, $\dot{\mathbf{v}} = (\mathbf{v}_{\text{target}} - \mathbf{v})/\tau_d$ with $\tau_d = 0.25$ s, in place of AirSim’s SimpleFlight.

3.5 The tunnel

`world.Tunnel(direction, seed)` builds a $5 \times 5 \times 50$ m tunnel along $+x$, $+y$ or $+z$ (the camera always faces $+x$), four half-space walls, and fifteen obstacles of three kinds in equal numbers: full sizes $\mathcal{U}[0.8, 1.8]$ m, positions along the tunnel in $[5, 47]$ m at least 2.5 m apart (the slack split by a Dirichlet draw), transverse offsets $\mathcal{U}[-1.6, 1.6]^2$; pillars span the cross-section along the tunnel’s second transverse axis. Collision uses the physical hull (cube, sphere, pillar) and a drone radius of 0.35 m; a collision is counted at contact onset, floor contacts excluded as in the study’s evaluation notes.

3.6 The operator model

`pilot.Pilot` replaces the participant. It perceives obstacles through three channels with separate delays: the front camera ($90^\circ \times 60^\circ$ field of view, 12 m range, delay $\tau_v = 0.35$ s, weight 1), the top-down inset of the study GUI (obstacles up to 4 m ahead along the tunnel, delay 0.8 s, weight 0.4),

and the cues (delay $\tau_h = 0.35$ s). It cruises at 2 m/s, aims at a pass-by point beside the nearest perceived obstacle chosen among two (pillar) or four (cube, sphere) candidates for the largest clearance from same-depth neighbours and walls, then the shorter side-step, with a proportional gain of 2 s^{-1} on the transverse error, brakes to 40 % when the obstacle is within 3 m and the side-step is incomplete, keeps 1.5 m from the walls, and pushes the stick away from the direction of every active actuator with duty ≥ 4 with a saturating gain of 1.0 m/s (braking to 65 % when a strong cue comes from ahead). Commands pass through a 0.15 s stick lag, Gaussian noise of 0.1 m/s per axis and a ± 5 m/s clamp. If progress stalls for 3 s it backs up for one second and re-plans. Table 1 lists the parameters; none was fitted to the paper’s numbers beyond the qualitative pattern (forward easiest, right and upward hardest).

Table 1: Operator model parameters (`pilot.Pilot` defaults).

cruise speed	2.0 m/s	stick clamp	5 m/s per axis
camera field of view	$90^\circ \times 60^\circ$, 12 m	inset range / weight	4 m / 0.4
visual delay τ_v	0.35 s	inset delay	0.8 s
haptic delay τ_h	0.35 s	haptic gain k_{hap} , threshold	1.0 m/s, duty ≥ 4
side-step gain k_p	2.0 s^{-1}	pass-by margin	0.8 m + half size
stick lag	0.15 s	stick noise	0.1 m/s
stall time / back-up	3 s / 1 s	drone lag τ_d , radius	0.25 s, 0.35 m

3.7 The closed loop and the metrics

`sim.fly` runs the study loop at 50 Hz: operator command $\rightarrow \mathbf{u}_{\text{ref}} \rightarrow$ global QP over the in-range constraints $\rightarrow$ MultiCBF cues $\rightarrow$ drone update $\rightarrow$ collisions, until the drone reaches 50 m along the tunnel or 60 s elapse. Per flight it records distance travelled, collisions, mean and median input disagreement, flight time, the fraction of frames with a cue, the minimum h , the mean angular error between the obstacle direction and the actuator that fired, and the activation count of every actuator. A flight is a deterministic function of (direction, seed, condition); the same tunnels are flown in every condition (paired design).

3.8 The actuator layout as data

`suit.paper_layout()` reads the released `ActuatorDirections_norm.txt`, converts each Unity direction (x, y, z) to the study’s world frame as the study code does (`[z, x, y]`: forward, right, up), and recovers the 32 motor ids with their bus. `tests` check that the directions are exactly the paper’s grid: $\vartheta \in \{30, 60, 90, 120\}^\circ$ and φ every 45° . For the figures and the 3-D suit, each direction is mapped to a body position by shooting its ray from the torso centre through a torso model (super-elliptic cross-sections, half-width 15 cm to 20.5 cm, half-depth 10.5 cm to 12.2 cm, height 70 cm) and stretching the exit height by 2.8 so that the four rings land on the shoulder line, chest, ribs and hips as in the paper’s Fig. 5. The exact garment positions were not released; the model reproduces the rings, not the millimetres. Alternative layouts for the sweep (`make_layout`): an 8-direction horizontal ring, 16 (two rings), 48 (six rings including the dropped ones), 96 Fibonacci points, and an exact (unquantised) direction.

4 Experiments

All numbers are means $\pm$ SEM over tunnels; 1332 flights in total. The campaign runs as shards (`run_study.py main/sweeps`) and is merged by `run_study.py merge`; the figures below are regenerated by `make_figures.py`.

4.1 Verification

`pytest -q tests` (6 tests, 7 s): analytic gradients and curvature terms of all three barrier types against central finite differences (10^{-5}); the exact QP against SLSQP on 30 random feasible problems; the closed-form local projection; the released layout (32 ids, the 4×8 grid, four buses); cues that fire only when the constraint is violated and select the unit facing the obstacle; and two full flights that finish the tunnel with and without cues.

4.2 One flight

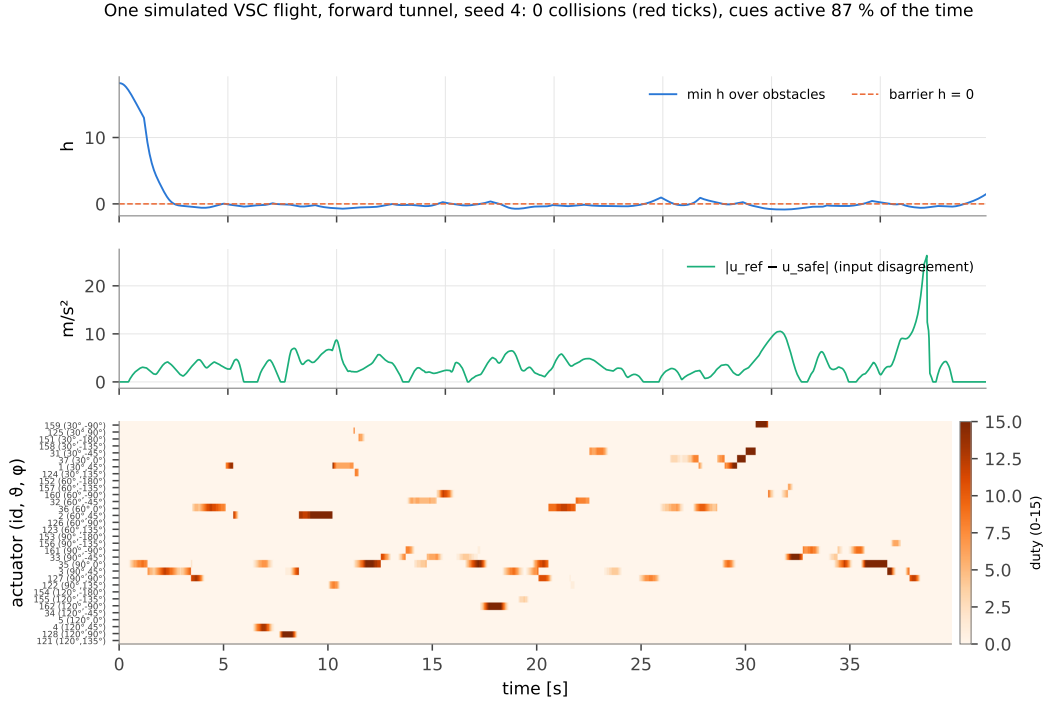


Figure 2: One VSC flight in the forward tunnel (seed 4): the minimum barrier value over obstacles with collisions as red ticks, the input disagreement, and the duty of each of the 32 units over time (rows ordered by ring and azimuth).

Figure 2 shows a typical cued flight: cues come in bursts of one to two seconds, one or two units at a time, whenever the path enters a shell or the input points into one; the shells are large (the padding plus the obstacle’s own size) so $h < 0$ is common and harmless while a collision needs the physical hull. Figure 3 shows the paired comparison on one tunnel.

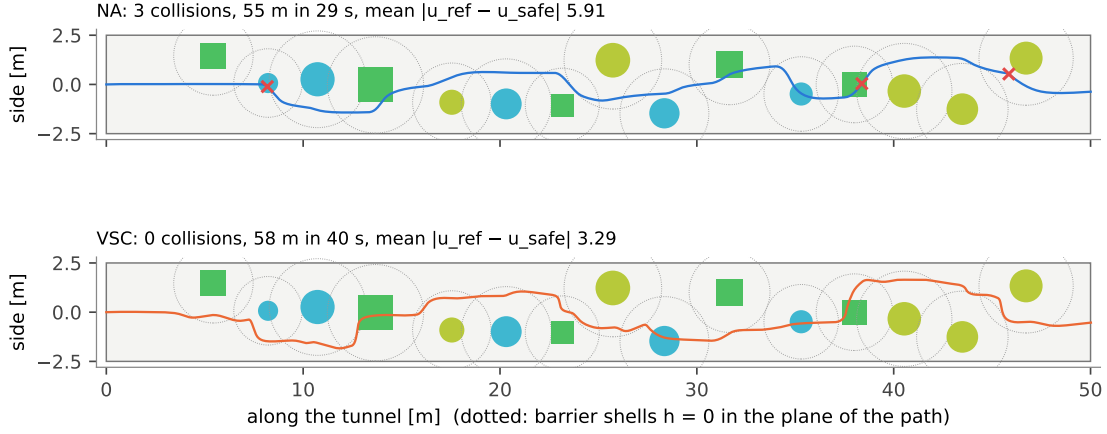


Figure 3: The same tunnel flown by the same operator model without feedback (top) and with the cues (bottom); crosses are collisions, dotted circles the barrier shells $h = 0$ in the plane of the path.

Table 2: Simulated study, 24 tunnels per direction, the same tunnels in both conditions.

condition	direction	collisions	distance [m]	$\ u_{\text{ref}} - u_{\text{safe}}\ $ [m/s ²]	time [s]	cue on [%]
no feedback	forward	2.92 ± 0.21	55.35 ± 0.20	5.13 ± 0.23	29.64 ± 0.13	0
no feedback	right	5.71 ± 0.32	54.69 ± 0.27	7.46 ± 0.47	31.02 ± 0.29	0
no feedback	upward	5.92 ± 0.35	54.73 ± 0.28	7.61 ± 0.49	31.26 ± 0.31	0
cues (VSC)	forward	0.54 ± 0.15	57.82 ± 0.28	3.01 ± 0.10	38.02 ± 0.18	80
cues (VSC)	right	0.67 ± 0.20	57.08 ± 0.28	3.87 ± 0.19	42.02 ± 0.38	85
cues (VSC)	upward	0.83 ± 0.26	57.19 ± 0.39	3.75 ± 0.17	42.45 ± 0.51	86
no feedback	all	4.85 ± 0.23	54.92 ± 0.15	6.73 ± 0.27	30.64 ± 0.17	0
cues (VSC)	all	0.68 ± 0.12	57.36 ± 0.19	3.54 ± 0.10	40.83 ± 0.32	84

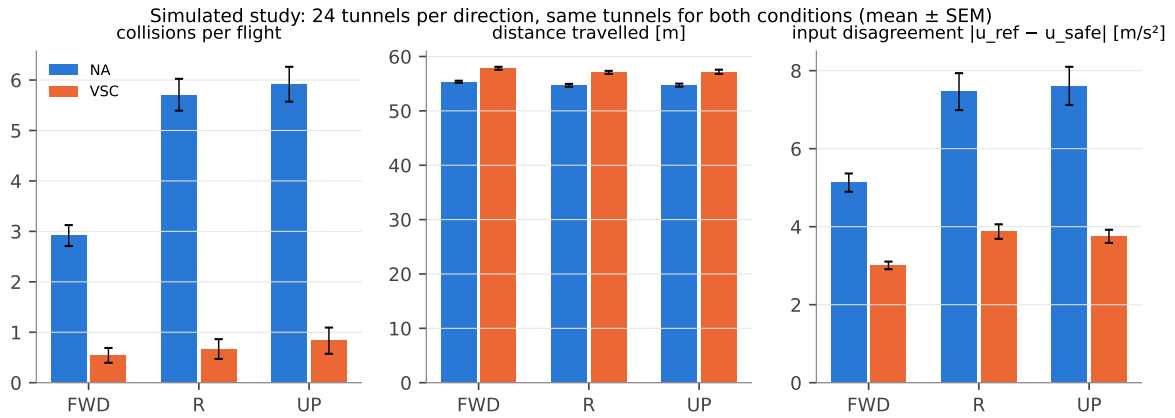


Figure 4: Collisions, distance and input disagreement by condition and direction (Table 2).

4.3 Main results

Table 2 and Figure 4 reproduce the paper’s qualitative pattern: without feedback the forward tunnel is the easiest (2.92 collisions) and the right and upward tunnels, flown with the camera pointing elsewhere, the hardest (5.71 and 5.92); the cues reduce collisions in every direction and the disagreement from 6.73 to 3.54. Two differences from the human study are expected and instructive. The reduction is much larger than the paper’s 7.5→5.2, because the modelled operator reacts to every strong cue after a fixed 0.35 s; Section 4.4 shows how fast this benefit disappears with slower reactions. And the cued flights are slower (40.83 against 30.64 s) because the model brakes when a cue comes from ahead, which the paper’s participants did not have to (their VSC distances were not longer than NA). The disagreement values are about twice the paper’s because the modelled operator penetrates the shells more deeply (minimum h down to -0.9), and a few frames deep inside a shell dominate the mean; the median (`disagreement_median` in `out/study.json`) is 25–35 % lower.

4.4 Sweeps

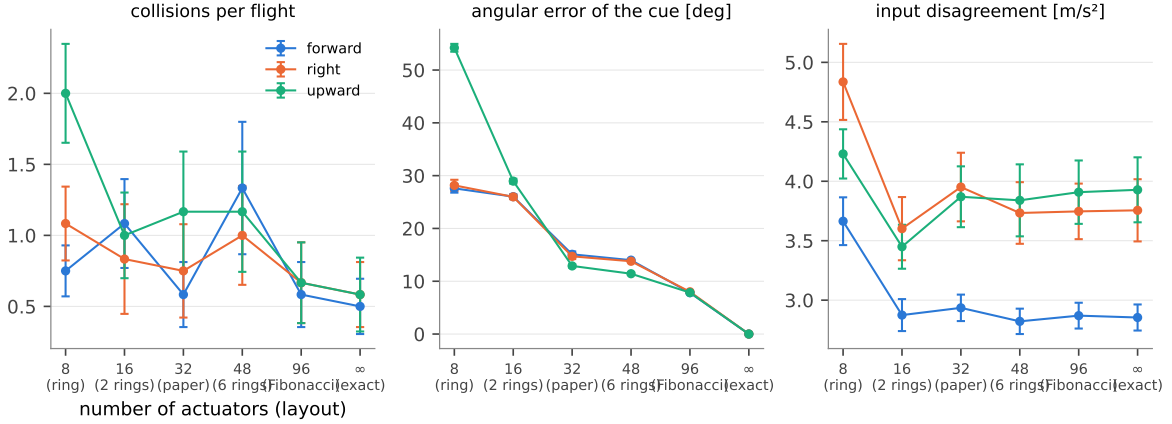


Figure 5: Actuator count (12 tunnels per direction, VSC): collisions, the angular error of the cue, and the disagreement, for an 8-direction ring, 16, the paper’s 32, 48 (six rings), 96 Fibonacci points and the exact direction.

Actuator count (Fig. 5). The mean angle between the obstacle direction and the unit that fires is 36.7° for 8 directions, 27.0° for 16, 14.2° for the paper’s 32, 13.1° for 48 and 7.9° for 96; collisions fall from 1.28 (8) to 0.97 (16), 0.83 (32), 0.64 (96) and 0.56 (exact). The 48-direction grid, which includes the ring the paper dropped, is not better (1.17): the extra directions point below the drone where this tunnel recipe has few obstacles, and the operator’s response to a lower-ring cue is a push upward that costs clearance elsewhere. Thirty-two directions capture most of the benefit of an exact direction in this model.

Reaction delay and gain (Fig. 6). With the paper’s $K_v = 3$, collisions are 0.42 at $\tau_h = 0.1$ s, 0.53 at 0.25 s, 1.14 at 0.5 s and 2.50 at 0.8 s, against 4.85 without feedback. A weak gain ($K_v = 1$: 1.69 to 4.00) loses most of the benefit at any delay because the duty rarely crosses the operator’s threshold; a strong gain ($K_v = 6$: 0.25 to 1.89) buys back part of what a slow operator loses. This is the single most sensitive knob in the study, and it belongs to the human, not to the algorithm.

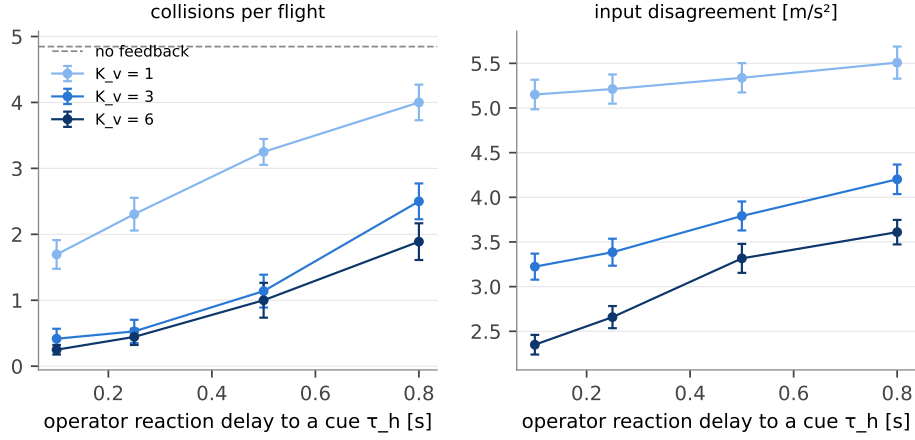


Figure 6: Operator reaction delay τ_h and intensity gain K_v (all directions pooled, 36 flights per point). Dashed: no feedback.

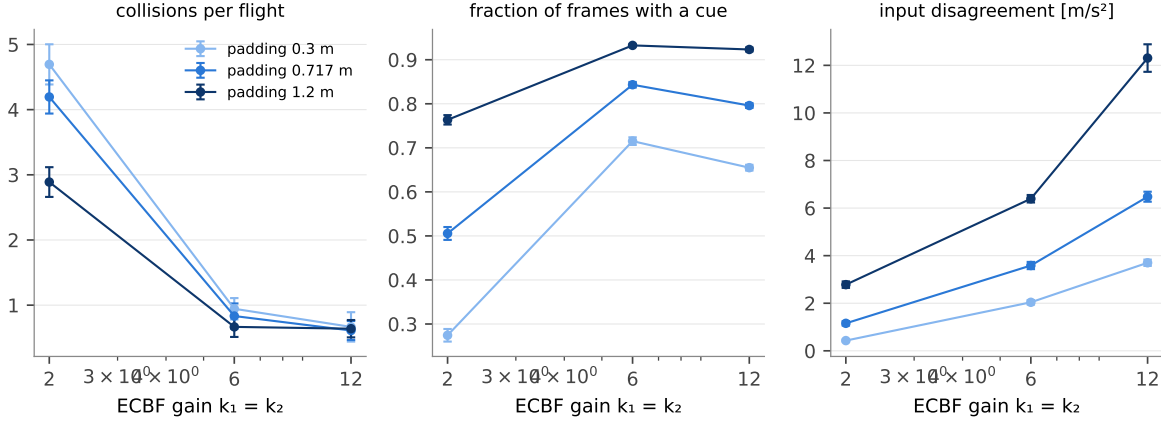


Figure 7: Barrier gains $k_1 = k_2$ and padding: collisions, fraction of frames with a cue, and disagreement.

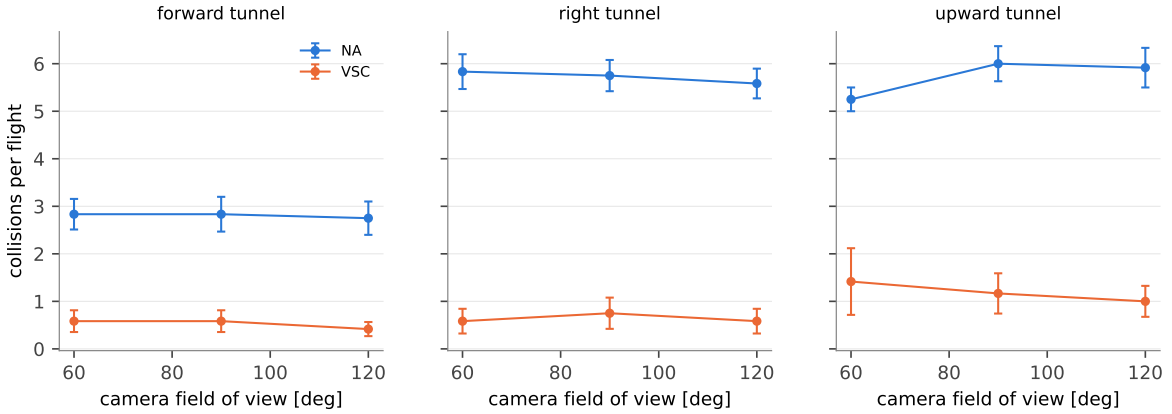


Figure 8: Camera field of view, no feedback against cues, per tunnel direction.

Barrier gains and padding (Fig. 7). With the paper’s padding, $k = 2$ fires the cue too late (4.19 collisions), $k = 6$ sits at the knee (0.83) and $k = 12$ adds little (0.61). More padding fires earlier and helps slightly (0.3 m: 0.94, 1.2 m: 0.67 at $k = 6$) but the disagreement grows steeply (2.04, 3.59, 6.39; 12.31 at $k = 12$ with 1.2 m) and a unit is on for most of the flight. The study’s constants are a sensible compromise for these tunnel densities.

Field of view (Fig. 8). Widening the front camera from 60° to 120° changes collisions little (no feedback: 4.64, 4.86, 4.75; cues: 0.86, 0.83, 0.67), because in the right and upward tunnels the obstacles lie outside any forward-facing view; what matters is where the camera points, which is the paper’s flying-direction manipulation.

4.5 Which units fire

Which actuators fire: the walls and the obstacles ahead drive the front ring in the forward tunnel; the side and the shoulder/hip rings take over when the tunnel turns right or up

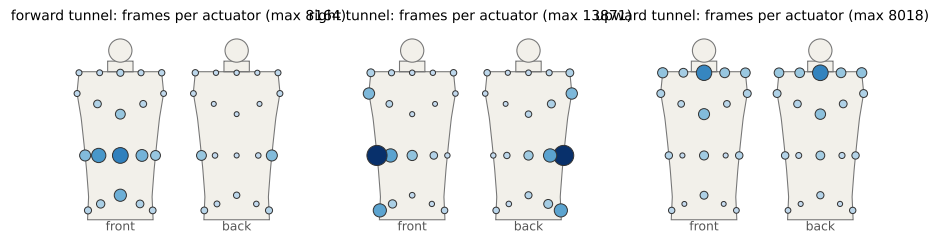


Figure 9: Frames each unit was driven, summed over the 24 cued flights per direction.

Figure 9: in the forward tunnel the front ring and the shoulder line fire (walls and obstacles ahead); in the right tunnel the right-side units of the ribs and shoulder rings; in the upward tunnel the shoulder line for what lies ahead along the climb and the lower back for what is below, which is the paper’s own remark that obstacles under the drone have to be mapped to the lower back.

5 Implementation and usage

Layout: experiments/aerohaptix/ (package: cbf.py, suit.py, world.py, pilot.py, sim.py), experiments/tests/, experiments/run_study.py, make_figures.py, make_videos.py, export_web.py; out/ (study.json, numbers.json, video/); docs/ (this report, figures/, numbers.tex). Dependencies: numpy, scipy (tests only), matplotlib, ffmpeg (videos).

```
cd experiments
python -m pytest -q tests
for k in 0 1 2; do python -u run_study.py main --seeds 24 --shard $k --nshards 3 & done
for k in 0 1 2 3; do python -u run_study.py sweeps --seeds 12 --shard $k --nshards 4 & done
python run_study.py merge # out/study.json + tables
python make_figures.py # docs/figures/*.pdf, site PNGs, results JSON,
# docs/numbers.tex
python make_videos.py --which all # out/video/*.mp4 (chase view, body map,
# top view, timeline)
python export_web.py # assets/data/aerohaptix_scene.json
cd ../docs && make
```

One flight costs about 1.8 ms per 50 Hz frame in Python (fifteen obstacles, four walls, exact QP). The web page carries a line-by-line JavaScript port (`assets/js/aerohaptix_cbf.js`) that drives an interactive 3-D suit, a barrier playground and the replays of the exported flights, and lets a visitor fly the tunnel with a keyboard or an Xbox controller through the same loop at 50 Hz. Hardware path: the cue rendering returns motor ids and duties per frame in the format of the study’s BLE module (`addr, mode, duty, freq, wave`), so the same loop can drive the real suit through its central unit.

6 Limitations

- The operator is a model. Every effect size on this page depends on its parameters (Table 1); the sweeps report the sensitivity, not a prediction for people. The paper’s human numbers are the reference.
- The simulator is not AirSim: a first-order velocity tracker and a hull-based collision model replace SimpleFlight’s dynamics and Unreal’s collision response. Distances and times are comparable in shape, not in value.
- The tunnel recipe (sizes, spacing) is a guess at the study’s; the paper does not list the obstacle dimensions. Collision counts scale with density.
- The body positions of the 32 units are from a torso model, not from the paper’s learned map; only the directions are the released ones.
- The velocity-increment convention of the released code is kept as is; a dimensionally consistent ECBF (input as acceleration, gains matched to the drone’s lag) would change the cue timing and is left for future work.
- Perception of vibration (masking between neighbouring units, the 16-level intensity scale) is not modelled; the operator responds to duty levels directly.

References

- [1] B. Huang, Z. Wang, Q. Cheng, S. Ren, H. Cai, A. Alvarez Valdivia, K. Mahadevan and D. Wigdor, “AeroHaptix: A wearable vibrotactile feedback system for enhancing collision avoidance in UAV teleoperation,” *IEEE Robotics and Automation Letters*, 2025; arXiv:2407.12105.
- [2] B. Huang, *DroneSimulation* (study code: AirSim client, ECBF classes, tactile module, actuator direction table), <https://github.com/huangbj16/DroneSimulation>.
- [3] D. Zhang, G. Yang and R. P. Khurshid, “Haptic teleoperation of UAVs through control barrier functions,” *IEEE Transactions on Haptics*, vol. 13, no. 1, pp. 109–115, 2020.
- [4] Q. Nguyen and K. Sreenath, “Exponential control barrier functions for enforcing high relative-degree safety-critical constraints,” *American Control Conference*, pp. 322–328, 2016.
- [5] A. D. Ames, X. Xu, J. W. Grizzle and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.

- [6] H. Elsayed, M. Weigel, F. Müller, M. Schmitz, K. Marky, S. Günther, J. Riemann and M. Mühlhäuser, “VibroMap: Understanding the spacing of vibrotactile actuators across the body,” *Proc. ACM IMWUT*, vol. 4, no. 4, 2020.
- [7] S. Shah, D. Dey, C. Lovett and A. Kapoor, “AirSim: High-fidelity visual and physical simulation for autonomous vehicles,” *Field and Service Robotics*, 2017.
- [8] S. G. Hart and L. E. Staveland, “Development of NASA-TLX (task load index): Results of empirical and theoretical research,” *Advances in Psychology*, vol. 52, pp. 139–183, 1988.
- [9] B. Huang, S. Ren, Y. Luo, Q. Cheng, H. Cai, Y. Sang, M. Sousa, P. H. Dietz and D. Wigdor, “VibraForge: A scalable prototyping toolkit for creating spatialized vibrotactile feedback systems,” *Proc. CHI*, 2025.